

Método guloso

Aula 10

Fábio Henrique Viduani Martinez

Faculdade de Computação
Universidade Federal de Mato Grosso do Sul

Projeto e Análise de Algoritmos I

Conteúdo da aula

- 1 Motivação
- 2 Problema da seleção de atividades
- 3 Códigos de Huffman
- 4 Elementos da estratégia gulosa
- 5 Exercícios

Método guloso

- ▶ algoritmos para problemas de otimização executam, em geral, uma sequência de passos, com um conjunto de escolhas em cada passo
- ▶ para muitos problemas de otimização, o uso da programação dinâmica para determinar as melhores escolhas é um exagero
- ▶ um algoritmo guloso sempre faz a escolha que parece melhor no momento
- ▶ ou seja, um algoritmo guloso faz uma escolha localmente ótima, na esperança que essa escolha levará a uma solução globalmente ótima
- ▶ algoritmos gulosos nem sempre fornecem soluções ótimas, mas o fazem para muitos problemas de otimização

Introdução

- ▶ problema de escalonamento de várias atividades que competem pelo uso exclusivo de um recurso comum, com objetivo de selecionar um conjunto de tamanho máximo de atividades compatíveis
- ▶ seja $S = \{a_1, a_2, \dots, a_n\}$ um conjunto de n **atividades** que desejam usar um recurso que pode ser usado por apenas uma atividade por vez
- ▶ cada atividade a_i tem um **tempo de início** s_i e um **tempo de término** f_i , onde $0 \leq s_i < f_i < \infty$
- ▶ se selecionada, a atividade a_i é executada durante o intervalo $[s_i, f_i)$
- ▶ atividades a_i e a_j são **compatíveis** se os intervalos $[s_i, f_i)$ e $[s_j, f_j)$ não se sobrepõem, isto é, se $s_i \geq f_j$ ou $s_j \geq f_i$

Problema

Problema da seleção de atividades

Dado um conjunto de n atividades $S = \{a_1, a_2, \dots, a_n\}$, selecionar um subconjunto de S de atividades mutuamente compatíveis, de tamanho máximo

Exemplos

Exemplo:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

Exemplos

Exemplo:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

$$\{a_3, a_9, a_{11}\}$$

Exemplos

Exemplo:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

$$\{a_1, a_4, a_8, a_{11}\}$$

Exemplos

Exemplo:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

$$\{a_2, a_4, a_9, a_{11}\}$$

Subestrutura ótima do problema

- ▶ como em programação dinâmica, devemos encontrar a subestrutura ótima do problema e então usá-la para construir uma solução ótima para o problema a partir das soluções ótimas para os subproblemas
- ▶ defina o subconjunto S_{ij} de atividades em S que podem ser iniciadas depois que a atividade a_i termina e que terminam antes que a atividade a_j inicia:

$$S_{ij} = \{a_k \in S : f_i \leq s_k < f_k \leq s_j\}$$

- ▶ adicionamos atividades fictícias a_0 e a_{n+1} ao conjunto S e fazemos $f_0 = 0$ e $s_{n+1} = \infty$
- ▶ $S = S_{0,n+1}$ e $0 \leq i, j \leq n+1$

Subestrutura ótima do problema

- ▶ suponha, sem perda de generalidade, que as atividades estão ordenadas por ordem monotonicamente crescente de tempos de término:

$$f_0 \leq f_1 \leq f_2 \leq \dots \leq f_n < f_{n+1}$$

- ▶ é fácil ver que $S_{ij} = \emptyset$ sempre que $i \geq j$
- ▶ portanto, supondo que as atividades estão ordenadas por ordem monotonicamente crescente de tempos de término, o espaço de subproblemas se restringe a selecionar um subconjunto de tamanho máximo de atividades mutuamente compatíveis de S_{ij} , para $0 \leq i < j \leq n+1$, já que todos os outros subconjuntos S_{ij} são vazios

Subestrutura ótima do problema

- ▶ considere um subproblema não vazio S_{ij} e suponha que uma solução para S_{ij} inclua a atividade a_k , tal que $f_i \leq s_k < f_k \leq s_j$
- ▶ o uso da atividade a_k gera dois subproblemas: S_{ik} e S_{kj} , que são subconjuntos de atividades de S_{ij}
- ▶ uma solução para S_{ij} é a união das soluções para S_{ik} e S_{kj} mais a atividade a_k
- ▶ o número de atividades em uma solução para S_{ij} é o tamanho da solução para S_{ik} , mais o tamanho da solução para S_{kj} , mais um

Subestrutura ótima do problema

- ▶ suponha que uma solução ótima A_{ij} para S_{ij} inclui a atividade a_k
- ▶ então, as soluções A_{ik} para S_{ik} e A_{kj} para S_{kj} usadas nessa solução ótima para S_{ij} devem também ser ótimas
- ▶ podemos construir um subconjunto de tamanho máximo de atividades mutuamente compatíveis em S_{ij} através da quebra desse problema em dois subproblemas (encontrar subconjuntos de tamanho máximo de atividades mutuamente compatíveis em S_{ik} e S_{kj}), encontrar subconjuntos de tamanho máximo A_{ik} e A_{kj} de atividades mutuamente compatíveis para esses subproblemas e compor um subconjunto de tamanho máximo A_{ij} de atividades mutuamente compatíveis da forma

$$A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$$

- ▶ uma solução ótima para o problema todo é uma solução para $S_{0,n+1}$

Solução recursiva

- ▶ seja $c[i, j]$ o número de atividades em um subconjunto de tamanho máximo de atividades mutuamente compatíveis em S_{ij}
- ▶ $c[i, j] = 0$ quando $S_{ij} = \emptyset$ ($i \geq j$)
- ▶ $c[i, j] = c[i, k] + c[k, j] + 1$ quando $S_{ij} \neq \emptyset$ ($i < j$)
- ▶ a definição recursiva completa para $c[i, j]$ é dada por:

$$c[i, j] = \begin{cases} 0, & \text{se } S_{ij} = \emptyset, \\ \max_{\substack{i < k < j \\ a_k \in S_{ij}}} \{c[i, k] + c[k, j] + 1\}, & \text{se } S_{ij} \neq \emptyset. \end{cases}$$

Algoritmo guloso

Teorema

Considere qualquer subproblema não vazio S_{ij} e seja a_m a atividade em S_{ij} com o menor tempo de término:

$$f_m = \min \{f_k : a_k \in S_{ij}\}.$$

Então,

1. Atividade a_m é usada em algum subconjunto de tamanho máximo de atividades mutuamente compatíveis de S_{ij} ;
2. O subproblema S_{im} é vazio e a escolha de a_m leva ao subproblema S_{mj} como o único subproblema que pode ser não vazio.

Algoritmo guloso

- ▶ o resultado anterior garante duas propriedades importantes:
 1. apenas um subproblema é usado em uma solução ótima (já que os outros subproblemas são vazios); e
 2. na solução do subproblema S_{ij} é necessário considerar somente uma escolha: aquela com o menor tempo de término em S_{ij}
- ▶ além disso, esse resultado também permite a solução de cada subproblema “de cima para baixo” (*top-down*)

Algoritmo guloso

RECURSIVE-ACTIVITY-SELECTOR(s, f, i, n)

```

01.   $m \leftarrow i + 1$ 
02.  enquanto  $m \leq n$  e  $s_m < f_i$ 
03.     $m \leftarrow m + 1$ 
04.  se  $m \leq n$ 
05.    devolva  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$ 
06.  senão
07.    devolva  $\emptyset$ 

```

Tempo de execução: $\Theta(n)$

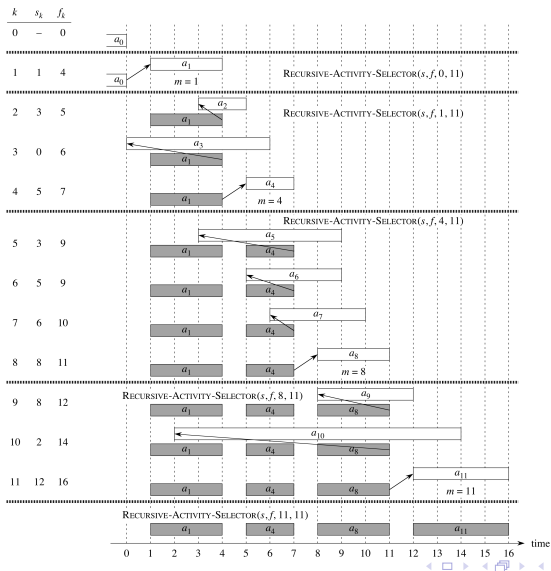
Algoritmo guloso

GREEDY-ACTIVITY-SELECTOR(s, f)

```
01.  $n \leftarrow s.length$ 
02.  $A \leftarrow \{a_1\}$ 
03.  $i \leftarrow 1$ 
04. para  $m \leftarrow 2$  até  $n$ 
05.     se  $s_m \geq f_i$ 
06.          $A \leftarrow A \cup \{a_m\}$ 
07.          $i \leftarrow m$ 
08. devolva  $A$ 
```

Tempo de execução: $\Theta(n)$

Algoritmo guloso



Introdução

- ▶ códigos de Huffman são amplamente usados e são uma técnica muito efetiva para compressão de dados
- ▶ a taxa de compressão varia de 20% a 90%, dependendo das características dos dados a comprimir
- ▶ o algoritmo guloso de Huffman usa uma tabela de frequências de ocorrências de caracteres que obtém uma forma ótima de representar cada caractere como uma cadeia de caracteres binários

Exemplo

- ▶ Suponha que temos um arquivo de dados com 100.000 caracteres que queremos armazenar compactamente
- ▶ As frequências com que os caracteres ocorrem nesse arquivo são dadas de acordo com a tabela a seguir:

	a	b	c	d	e	f
Frequência (em milhares)	45	13	12	16	9	5

- ▶ Apenas seis caracteres diferentes ocorrem no arquivo e o caractere **a**, por exemplo, aparece 45.000 vezes

Código binário

- ▶ consideramos o problema de projetar um **código de caracteres binários** (ou simplesmente **código**), onde cada caractere é representado por uma única cadeia de caracteres binários
- ▶ se usamos um **código de comprimento fixo**, precisamos de três bits para representar seis caracteres:
 $a = 000, b = 001, c = 010, d = 011, e = 100, f = 101$
- ▶ esse método necessita de 300.000 bits para codificar o arquivo inteiro
- ▶ podemos fazer melhor?

Código binário

- ▶ um **código de comprimento variável** fornece códigos binários mais curtos para caracteres mais frequentes e mais longos para caracteres menos frequentes
- ▶ por exemplo, **a** = 0, **b** = 101, **c** = 100, **d** = 111, **e** = 1101, **f** = 1100
- ▶ esse código necessita de

$$(45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4) \cdot 1000 = 224.000 \text{ bits}$$

economizando cerca de 25%

- ▶ esse código é ótimo

Exemplo

	a	b	c	d	e	f
Frequência (em milhares)	45	13	12	16	9	5
Código de comprimento fixo	000	001	010	011	100	101
Código de comprimento variável	0	101	100	111	1101	1100

Códigos livres de prefixos

- ▶ um **código livre de prefixos**, ou simplesmente **código prefixo**, é aquele tal que nenhuma cadeia de caracteres binários do código é também um prefixo de alguma outra cadeia de caracteres binários do código
- ▶ é possível mostrar que a compressão de dados ótima obtida por um código pode sempre ser obtida por um código prefixo
- ▶ a codificação para um código de caracteres binários é sempre muito simples: basta concatenar os códigos que representam cada caractere do arquivo de dados
- ▶ por exemplo, usando o código prefixo de comprimento variável do exemplo acima, codificamos o arquivo de dados **abc** como $0 \cdot 101 \cdot 100 = 0101100$, onde usamos o símbolo “.” para denotar concatenação

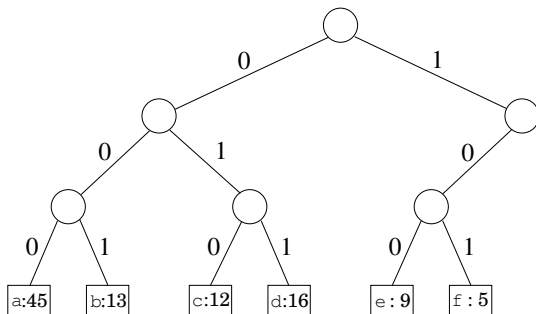
Códigos livres de prefixos

- ▶ a decodificação é simplificada com o uso de um código prefixo
- ▶ como nenhum código é prefixo de qualquer outro, o código que começa um arquivo codificado não é ambíguo
- ▶ podemos então identificar o código inicial, traduzi-lo para o caractere original e repetir a decodificação no arquivo codificado restante
- ▶ por exemplo, a cadeia 001011101 é decodificada como 0·0·101·1101, que é traduzida em **aabe**

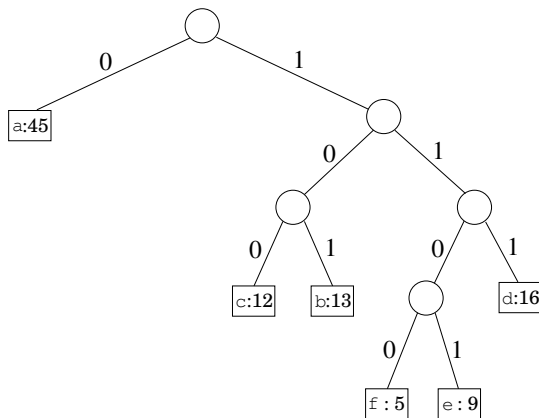
Códigos livres de prefixos

- ▶ no processo de decodificação, representamos convenientemente o código prefixo como uma árvore binária cujas folhas são os caracteres dados
- ▶ interpretamos o código binário para um caractere como o caminho da raiz até o caractere, onde 0 significa “vá para o filho esquerdo” e 1 significa “vá para o filho direito”

Códigos livres de prefixos



Códigos livres de prefixos



Códigos livres de prefixos

- ▶ essas árvores não são árvores binárias de busca
- ▶ um código ótimo para um arquivo é sempre representado por uma árvore binária cheia, onde todo vértice interno tem dois filhos

Códigos livres de prefixos

- ▶ se C é o alfabeto onde os caracteres estão contidos e todas as frequências de caracteres são positivas, então a árvore para um código prefixo ótimo tem exatamente $|C|$ folhas, uma para cada símbolo do alfabeto, e $|C| - 1$ vértices internos
- ▶ dada uma árvore T correspondente a um código prefixo, o número de bits de um arquivo codificado é dado da seguinte forma: para cada caractere c no alfabeto C , denote por $c.freq$ a frequência de c no arquivo e denote por $d_T(c)$ a profundidade da folha contendo c na árvore T
- ▶ o número de bits necessários para codificar um arquivo, também chamado de **custo** de T , é então

$$B(T) = \sum_{c \in C} c.freq \cdot d_T(c)$$

Algoritmo de Huffman

- ▶ o **código de Huffman** é um algoritmo guloso que constrói um código prefixo ótimo
- ▶ no código de Huffman, C é um conjunto de n caracteres e cada caractere $c \in C$ é um objeto com uma frequência definida $c.freq$
- ▶ o algoritmo constrói a árvore T correspondente ao código ótimo “de baixo para cima”
- ▶ inicia com um conjunto de $|C|$ folhas e executa uma sequência de $|C| - 1$ operações de “união” para obter a árvore final
- ▶ usa uma lista de min-prioridades Q , com chave f , para identificar os dois objetos menos frequentes e então uni-las

Algoritmo de Huffman

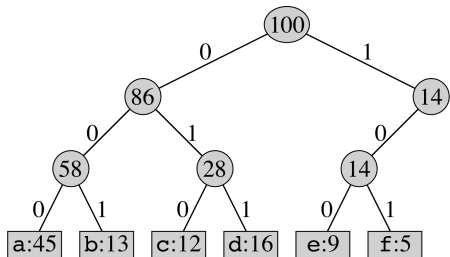
```

HUFFMAN(C)
01.   $n \leftarrow |C|$ 
02.   $Q \leftarrow C$ 
03.  para  $i \leftarrow 1$  até  $n - 1$ 
04.      aloque um novo vértice  $z$ 
05.       $z.left \leftarrow x \leftarrow \text{EXTRACT-MIN}(Q)$ 
06.       $z.right \leftarrow y \leftarrow \text{EXTRACT-MIN}(Q)$ 
07.       $z.freq \leftarrow x.freq + y.freq$ 
08.      INSERTLISTMIN( $Q, z$ )
09.  devolva  $\text{EXTRACT-MIN}(Q)$ 

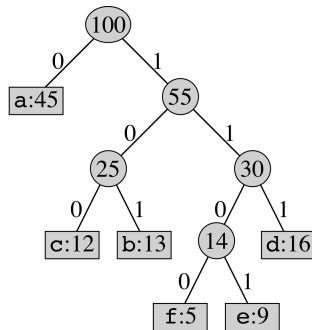
```

Tempo de execução: $O(n \lg n)$

Algoritmo de Huffman



(a)



(b)

Características do método guloso

- ▶ um algoritmo guloso obtém uma solução ótima para um problema fazendo uma sequência de escolhas
- ▶ para cada decisão a ser tomada no algoritmo, a decisão que parece a melhor naquele momento é escolhida
- ▶ essa estratégia nem sempre produz uma solução ótima, mas algumas vezes sim

Características do método guloso

o processo para desenvolver um algoritmo guloso segue os seguintes passos:

1. foque no problema de otimização como um problema em que fazemos uma escolha e então nos resta um único subproblema a ser resolvido
2. prove que sempre existe uma solução ótima do problema original que faz a escolha gulosa, de forma que a escolha gulosa é sempre segura
3. mostre que, tendo feito a escolha gulosa, o que resta é um subproblema com a propriedade que se combinamos uma solução ótima para o subproblema com a escolha gulosa que fizemos, obtemos então uma solução ótima para o problema original

Características do método guloso

- ▶ como alguém pode afirmar se um algoritmo guloso poderá resolver um problema de otimização?
 - ▶ **propriedade da escolha gulosa**: uma solução globalmente ótima pode ser obtida por meio de escolhas (gulosas) localmente ótimas
 - ▶ **subestrutura ótima**: uma solução ótima para o problema contém internamente soluções ótimas para seus subproblemas

Método guloso versus programação dinâmica

- ▶ a propriedade da subestrutura ótima é explorada pelo método guloso e pelo método da programação dinâmica
- ▶ assim, podemos tentar construir uma solução usando programação dinâmica para um problema, quando a solução gulosa é suficiente
- ▶ ou podemos pensar erroneamente que uma solução gulosa funciona bem, quando de fato a solução por programação dinâmica é necessária

Método guloso versus programação dinâmica

Problema da mochila binária

- ▶ um ladrão está roubando uma loja e encontra n objetos
- ▶ o i -ésimo objeto tem valor v_i e pesa w_i , onde v_i e w_i são números inteiros
- ▶ o ladrão quer levar a carga mais valiosa possível, mas ele pode carregar no máximo W quilos em sua mochila, para algum número inteiro W
- ▶ quais objetos ele deve levar?
- ▶ esse problema é chamado de **problema da mochila binária (ou 0-1)**, porque cada objeto deve ser levado ou deixado para trás

Método guloso versus programação dinâmica

Problema da mochila fracionária

- ▶ a situação é a mesma, mas o ladrão pode levar frações dos materiais, ao invés de ter de fazer uma escolha binária (0-1) para cada objeto
- ▶ esse problema é chamado de **problema da mochila fracionária**, porque o ladrão pode levar uma fração de cada material

Método guloso versus programação dinâmica

- ▶ ambos problemas exibem a propriedade da subestrutura ótima
- ▶ no problema da mochila binária, considere a carga total mais valiosa que pesa no máximo W quilos; se removemos o objeto j dessa carga, a carga restante deve ser a mais valiosa com peso no máximo $W - w_j$ que o ladrão pode levar dos $n - 1$ objetos originais, excluindo o objeto j
- ▶ no problema da mochila fracionária, se removemos w quilos de um material j de uma carga total ótima, a carga restante deve ser a mais valiosa carga com peso no máximo $W - w$ que o ladrão pode levar dos itens originais, mais $w_j - w$ quilos do material j
- ▶ apesar dos problemas serem similares, o problema da mochila fracionária pode ser solucionado pela estratégia gulosa, enquanto que o problema da mochila binária é solucionado por programação dinâmica

Método guloso versus programação dinâmica

- ▶ no problema da mochila fracionária, o ladrão deve primeiro computar o valor por quilo v_i/w_i de cada material
- ▶ obedecendo a escolha gulosa, o ladrão começa levando tanto quanto possível do material com maior valor por quilo
- ▶ se termina o suprimento desse material e o ladrão ainda pode roubar mais, ele leva tanto quanto possível do material com o próximo maior valor por quilo e assim por diante, até que ele não possa mais levar nenhum material
- ▶ assim, pela ordenação dos materiais pelo valor por quilo, o algoritmo tem tempo de execução $O(n \lg n)$

Mochila fracionária

```

FRACTIONALKNAPSACK( $v, w, W$ )
01.  ordene  $v$  e  $w$  por  $v_i/w_i$ 
02.  para  $i \leftarrow 1$  até  $v.length$ 
03.       $x_i \leftarrow 0$ 
04.   $i \leftarrow 1$ 
05.  enquanto  $w_i \leq W$ 
06.       $x_i \leftarrow 1$ 
07.       $W \leftarrow W - w_i$ 
08.       $i \leftarrow i + 1$ 
09.  se  $W > 0$ 
10.       $x_i \leftarrow W/w_i$ 
11.  devolva  $x$ 

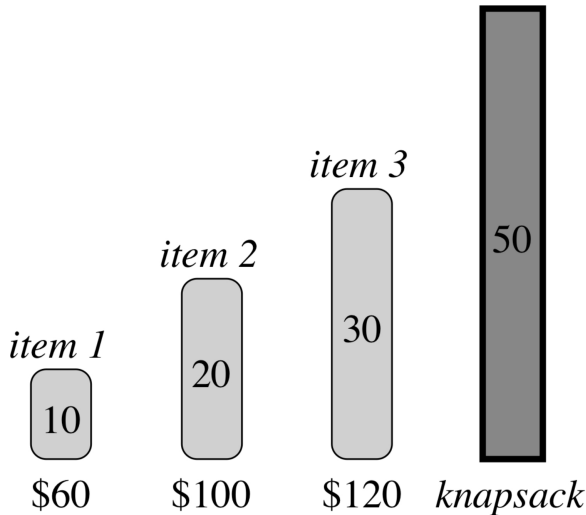
```

Tempo de execução: $O(n \lg n)$

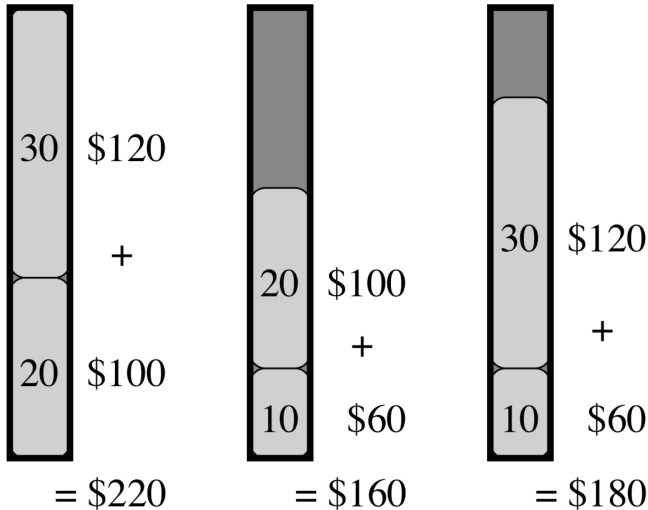
Método guloso versus programação dinâmica

a estratégia gulosa funciona para o problema da mochila binária?

Método guloso versus programação dinâmica



Método guloso versus programação dinâmica



Método guloso versus programação dinâmica

$\frac{20}{30}$ \$80
+
20 \$100
+
10 \$60
= \$240

Mochila binária

- ▶ quando consideramos um objeto no problema da mochila binária, devemos comparar a solução para o subproblema em que esse objeto é incluído contra a solução para o subproblema em que esse objeto é excluído, antes de fazer a escolha
- ▶ a formulação do problema dessa forma resulta em muitos subproblemas sobrepostos, uma característica fundamental da programação dinâmica

Exercícios

- 16.1-1 Forneça um algoritmo de programação dinâmica para o problema da seleção de atividades, baseado na recorrência vista em sala, isto é, seu algoritmo deve computar os tamanhos $c[i, j]$ como definido pela recorrência e também determinar um subconjunto de tamanho máximo de atividades mutuamente compatíveis. Considere que as atividades de entrada foram ordenadas crescentemente pelos seus tempos de término. Compare o tempo de execução de sua solução com o tempo de execução do algoritmo GREEDY-ACTIVITY-SELECTOR.
- 16.1-2 Suponha que ao invés de sempre selecionar a atividade que termina mais cedo, selecionamos a atividade que inicia mais tarde e que é compatível com todas as outras atividades previamente selecionadas. Descreva como esta estratégia é um algoritmo guloso e mostre que ela fornece uma solução ótima.

Exercícios

- 16.1-3 Suponha que temos um conjunto de atividades para escalonar entre um grande número de auditórios. Desejamos escalonar todas as atividades usando o menor número possível de auditórios. Forneça um algoritmo guloso eficiente para determinar quais atividades devem utilizar quais auditórios.
- 16.1-4 Nem toda estratégia gulosa para o problema da seleção de atividades produz um conjunto de tamanho máximo de atividades mutuamente compatíveis. Forneça um exemplo para mostrar que a estratégia de selecionar a atividade de menor duração dentre aquelas que são compatíveis com as atividades previamente selecionadas não funciona. Faça o mesmo para a estratégia de sempre selecionar a atividade compatível que possui a menor sobreposição sobre as atividades restantes e para a estratégia de sempre selecionar a atividade compatível com menor tempo de início.

Exercícios

16.1-5 Considere uma modificação do problema da seleção de atividades em que cada atividade a_i tem, além de seu tempo de início e término, um valor v_i . O objetivo não é apenas maximizar o número de tarefas escalonadas, mas maximizar o valor total das atividades escalonadas. Isto é, desejamos escolher um conjunto A de atividades compatíveis tal que $\sum_{a_k \in A} v(a_k)$ é maximizada. Forneça um algoritmo de tempo polinomial para este problema.

Exercícios

- 16.2-1 Prove que o problema da mochila fracionária tem a propriedade da escolha ótima
- 16.2-2 Escreva um algoritmo de programação dinâmica para solucionar o problema da mochila binária com tempo de execução $O(nW)$, onde n é o número de itens e W é o peso máximo dos itens que o ladrão pode colocar na sua mochila.
- 16.2-4 Professor Marcelo dirige seu carro de Natal a Campo Grande através da rodovia BR-266. O tanque de gasolina de seu carro, quando cheio, contém gasolina suficiente para viajar n quilômetros e seu mapa fornece as distâncias entre os postos de gasolina no seu caminho. O professor deseja fazer o menor número possível de paradas durante sua viagem. Forneça um método eficiente pelo qual o professor Marcelo pode determinar em quais postos de gasolina ele deve parar e argumente de alguma forma que sua estratégia fornece uma solução ótima.

Exercícios

- 16.2-5 Escreva um algoritmo eficiente que, dado um conjunto $\{x_1, x_2, \dots, x_n\}$ de pontos em uma reta real, determina o menor conjunto de intervalos fechados de comprimento unitário que contêm todos os pontos dados. Argumente que seu algoritmo está correto.
- 16.3-2 Prove que uma árvore binária que não é cheia não pode corresponder a um código livre de prefixos ótimo.
- 16.3-3 Qual seria um código de Huffman ótimo para o seguinte conjunto de frequências, baseadas nos primeiros 8 números de Fibonacci?

a : 1 b : 1 c : 2 d : 3 e : 5 f : 8 g : 13 h : 21

Você poderia generalizar sua resposta para obter um código ótimo quando as frequências são os primeiros n números de Fibonacci?

Exercícios

- 16.3-5 Mostre que se arranjamos os caracteres de um alfabeto de tal forma que suas frequências estejam em ordem decrescente, então existe um código ótimo cujos comprimentos das palavras codificadas são crescentes.
- 16.3-7 Generalize o algoritmo de Huffman para palavras de código ternário, isto é, palavras de código usando os símbolos 0, 1 e 2. Argumente que o algoritmo produz um código ternário ótimo.
- 16.3-8 Suponha que um arquivo de dados contenha uma sequência de caracteres de 8 bits tais que todos os 256 caracteres possuam uma característica em comum: a frequência máxima de qualquer caractere é menor que duas vezes a frequência mínima. Mostre que o código de Huffman, neste caso, não é mais eficiente que o uso de um código de tamanho fixo usual, ou seja, o código `ASCII` de 8 bits.

Problemas

16-1 Troco em moedas

Considere o problema de fazer o troco para n centavos usando o menor número de moedas. Assuma que cada valor de uma moeda é um número inteiro.

- Descreva um algoritmo guloso que faz o troco consistindo de moedas de 1, 5, 10 e 25 centavos de real. Prove que seu algoritmo fornece uma solução ótima.
- Suponha que os valores das moedas disponíveis são potências de c , isto é, são valores $c^0, c^1, \dots, c^k$ para inteiros $c > 1$ e $k \geq 1$. Mostre que um algoritmo guloso para fazer o troco sempre fornece uma solução ótima.
- Forneça um conjunto de valores de moedas para o qual o algoritmo guloso para fazer o troco não fornece uma solução ótima. Você deve incluir a moeda de 1 centavo de tal forma que existe uma solução para o problema para todo valor n .
- Forneça um algoritmo de tempo de execução $O(nk)$ que faz o troco para qualquer conjunto de k valores diferentes de moedas, considerando que uma das moedas vale 1 centavo.